PROJECT PHASE-I REPORT

AI-POWERED SECURE DECENTRALIZED FILE SHARING SYSTEM USING BLOCKCHAIN AND IPFS

Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of the

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

by

Muhammed Hafis (VAS23CS138)



(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Department of Computer Science and Engineering
Vidya Academy of Science & Technology
Thalakkottukara, Thrissur – 680 501
http://www.vidyaacademy.ac.in

October 2026

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Vidya Academy of Science & Technology

Thalakkottukara, Thrissur – 680 501

Department of Computer Science and Engineering

Certificate

This is to certify that the Main Project Phase I report titled **AI-Powered Secure Decentralized File Sharing System Using Blockchain and IPFS** is a bonafide record of the work carried out by **Muhammed Hafis (VAS23CS138)** of Vidya Academy of Science & Technology, Thrissur, in partial fulfilment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering** of **APJ Abdul Kalam Technological University**, during the academic year 2023-2027. The report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Project Guide

Dr. Sajay K R
Professor and Head, Dept. of CSE

Head of Department

Dr. Sajay K R
Head, Dept. of CSE

Project Co-guide

Mr. Muhammad Alishihab V S
Assistant Professor, Dept. of CSE

Project Coordinators

Ms. Anitha L
Assistant Professor, Dept. of CSE

Ms. Neethu George
Assistant Professor, Dept. of CSE

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Vidya Academy of Science & Technology

Thalakkottukara, Thrissur – 680 501

Department of Computer Science and Engineering

UNDERTAKING

I, **Muhammed Hafis (Univ. Reg. No. VAS23CS138)**, hereby undertake that the Main Project Phase I report titled **AI-Powered Secure Decentralized File Sharing System Using Blockchain and IPFS** is based on the work carried out by our project team under the guidance of **Dr. Sajay K R** (Professor and Head, Dept. of CSE) and **Mr. Muhammad Alishihab V S** (Assistant Professor, Dept. of CSE), Vidya Academy of Science & Technology, Thrissur, in partial fulfilment of the requirements for the award of the **Degree of Bachelor of Technology in Computer Science and Engineering** of **APJ Abdul Kalam Technological University**, during the academic year 2023-2027.

Place: Thrissur
Date: October 2026

Muhammed Hafis
Reg. No. VAS23CS138

Vision and Mission

Vidya Academy of Science & Technology

Vision

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Mission

M1: Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.

M2: Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.

M3: Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Department of Computer Science and Engineering

Vision

To excel in computer science and engineering education and research, nurturing innovative and ethically responsible professionals for sustainable technological development.

Mission

M1: Impart quality education in computer science and engineering with strong theoretical foundations, practical skills, and exposure to emerging technologies.

M2: Promote research, innovation, entrepreneurship, and industry collaboration for sustainable computing solutions.

M3: Develop professionally competent graduates with ethical integrity, leadership qualities, and social responsibility for lifelong learning and societal service.

Program Educational Objectives (PEOs)

PEO-1: Graduates will apply foundations of computer science and engineering to design and develop reliable software solutions using modern tools.

PEO-2: Graduates will pursue research, innovation, and entrepreneurship, engaging in lifelong learning and higher studies to create sustainable and impactful technological advancements.

PEO-3: Graduates will demonstrate ethical integrity, leadership, effective communication, and multidisciplinary teamwork, contributing responsibly to societal and technological progress.

Program Specific Outcomes (PSOs)

PSO-1: Build robust software systems and intelligent applications using modern programming languages, frameworks, and algorithms.

PSO-2: Apply core concepts in networking, security, and cloud computing to design efficient and secure systems.

PSO-3: Develop innovative and socially relevant computing solutions through continuous learning and collaboration.

Program Outcomes (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/ develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

ACKNOWLEDGEMENT

During the course of our Phase-I project work, several persons collaborated directly and indirectly with us. Without their support, it would have been impossible for us to finish our work. For this reason, we wish to use this section to acknowledge their support.

We want to start by expressing our thanks to our project guide, **Dr. Sajay K R**, Professor and Head, Dept. of CSE, and project co-guide, **Mr. Muhammad Alishihab V S**, Assistant Professor, Dept. of CSE, for their valuable advice and guidance towards this work. We received motivation, encouragement and support during the course of the work.

We would like to express our sincere gratitude to the project coordinators, **Ms. Anitha L** (Assistant Professor, Dept. of CSE) and **Ms. Neethu George** (Assistant Professor, Dept. of CSE), for their valuable suggestions and feedback.

We are grateful to all the faculty members of our department for their support, and We thank all our friends for their help and support in this work.

We are thankful to **Dr. Sajay K R**, Head of the Department of Computer Science and Engineering, and our Principal, **Dr. Sunitha C.**, for their kind co-operation.

Finally, we wish to express our gratitude to God Almighty for His abundant blessings, without which this effort would not have been successful.

Place: Thrissur
Date: October 2026

Muhammed Hafis
Reg. No. VAS23CS138

Seventh Semester B.Tech (2023 Admissions)
Vidya Academy of Science & Technology

ABSTRACT

In the modern digital environment, files are continuously shared across users, devices, teams and networks. Traditional centralized file-sharing platforms may expose sensitive information when servers or accounts are compromised, and basic upload workflows may accept malicious or incomplete files. This project presents Aegis Cyber Shield, an AI-powered secure decentralized file-sharing system that combines Zero Trust access control, AI-assisted threat screening, hybrid encryption, decentralized storage and blockchain-based integrity records in a single workflow. Every request is verified through a six-stage Zero Trust gateway covering token verification, IP allow-lists, session limits, role and attribute policies, behavioural scoring and AI risk assessment. Uploaded files are inspected by an AI security engine for malware indicators, high entropy, sensitive personal data and hidden content. Accepted files are encrypted using AES-256-GCM, with per-file keys wrapped using RSA-4096-OAEP. The encrypted content is pinned to IPFS through Pinata and identified by a Content Identifier (CID), while SHA-256 hashes, signatures and access permissions are registered in a Solidity smart-contract registry. Provenance records, audit logs and compliance monitoring support traceability and investigation. This Phase I report presents the requirements, literature survey, architecture, data flow, methodology, current implementation status and testing plan for the proposed system. Its effectiveness will be assessed through the planned functional, security and performance tests.

Keywords: secure file sharing, blockchain, IPFS, Zero Trust, hybrid encryption, AI threat detection

Contents

CERTIFICATE

UNDERTAKING

VISION, MISSION & PROGRAM EDUCATIONAL OBJECTIVES (PEO) **i**

PROGRAM OUTCOMES (PO) & PROGRAM SPECIFIC OUTCOMES (PSO) **ii**

ACKNOWLEDGEMENT **iii**

ABSTRACT **iv**

LIST OF FIGURES **vii**

LIST OF SYMBOLS AND ABBREVIATIONS **viii**

POs & PSOs ADDRESSED WITH JUSTIFICATION **x**

SUSTAINABLE DEVELOPMENT GOALS (SDG) **xiii-xiv**

SDGs ADDRESSED WITH JUSTIFICATION **xv**

1 INTRODUCTION **1**

1.1 General 1

1.2 Objective of the Work 2

1.3 Motivation for this Work 3

1.4 Methodology of the Project 4

2 LITERATURE REVIEW **7**

2.1 LITERATURE SURVEY 7

2.1.1 Zero Trust Architecture 7
2.1.2 IPFS – Content Addressed, Versioned, P2P File System 8
2.1.3 How to Time-Stamp a Digital Document 8
2.1.4 Blockchain-Enabled Intrusion Detection and Prevention System of APTs within Zero Trust Architecture 9
2.1.5 Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains 10
2.1.6 Applications of Blockchains in the Internet of Things: A Comprehensive Survey 10
2.1.7 The Role of Machine Learning in Cybersecurity 11
2.1.8 A Mathematical Theory of Communication 11
2.1.9 Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization 12
2.1.10 Keystroke Dynamics as a Biometric for Authentication 12
2.1.11 Security Operations Center: A Systematic Study and Open Challenges 13
2.1.12 Getting Physical with the Digital Investigation Process 13
2.2 Standards and Official Guidance Reviewed 13
2.3 Research Gap 15

3 SYSTEM DESIGN 16
3.1 System Architecture 16
3.1.1 End Users and Actors 17
3.1.2 Presentation Layer 18
3.1.3 Zero Trust Gateway 18
3.1.4 AI Security Engine 19
3.1.5 Hybrid Encryption Module 19
3.1.6 Blockchain Integrity Layer 19
3.1.7 Decentralized Storage Layer 20
3.1.8 Database and Data Models 20
3.1.9 File Sharing Module 21
3.1.10 Provenance and Audit Module 21
3.1.11 Agent Swarm, Threat Oracle and MCP Tools 21

3.1.12 SOC Dashboard and Compliance Monitoring 22
3.2 Use Cases 22
3.3 Security Controls and Compliance Mapping 23
3.4 API Surface 23

4 METHODOLOGY **25**
4.1 Requirement Analysis 25
4.2 System Design 25
4.3 Zero Trust Access Control 26
4.4 AI-Based File Screening 27
4.5 Hybrid Encryption and Integrity Verification 27
4.6 Decentralized Storage and Blockchain Registration 28
4.7 System Integration and Automation 28
4.8 Validation and Testing 29
4.9 Performance Evaluation and Optimization 29
4.10 Project Schedule 30

5 DATA FLOW **32**
5.1 Description 32
5.2 File Upload Sequence 33
5.3 File Download Sequence 33
5.4 File Sharing Flow 34

6 WORK FLOW **36**
6.1 Description 36

7 SYSTEM REQUIREMENTS **38**
7.1 **Hardware Requirements** 38
7.1.1 Processor 38
7.1.2 Random Access Memory (RAM) 38
7.1.3 Storage 39
7.1.4 Networking Requirements 39
7.1.5 Client-Side Requirements 40
7.2 **Software Requirements** 40

7.2.1 Operating System 40
7.2.2 Frontend Technologies 40
7.2.3 Backend Technologies 40
7.2.4 AI Engine 41
7.2.5 Blockchain and Storage 41
7.2.6 Development and Deployment Tools 41

8 EXPECTED RESULT **42**
8.1 Expected result 42
8.2 Implementation Progress 42
8.3 System Implementation Results 43
8.3.1 Web Interface Implementation 43
8.4 Risks and Limitations 47

9 CONCLUSION AND FUTURE WORK **49**
9.1 Conclusion 49
9.2 Future Scope of Work 50

BIBLIOGRAPHY **52**

List of Figures

3.1 System Architecture of Aegis Cyber Shield 16
3.2 Zero Trust Gateway Pipeline 18
3.3 Hybrid Encryption: Upload and Download Paths 20

5.1 Data Flow Diagram 32
5.2 File Upload Sequence 34
5.3 File Download Sequence 35

6.1 Work Flow Diagram 36

8.1 Login and Registration Page 44
8.2 User Dashboard 44
8.3 File Upload with AI Scan Result 45
8.4 Encrypted File List with CID and Hash 45
8.5 Share Dialog with Expiry and QR Link 46
8.6 Shared Inbox 46
8.7 SOC Dashboard and Threat Map 47
8.8 Provenance Timeline and Compliance Scorecard 47

List of Abbreviations

AES	Advanced Encryption Standard
ABAC	Attribute-Based Access Control
AI	Artificial Intelligence
API	Application Programming Interface
APT	Advanced Persistent Threat
CID	Content Identifier
DID	Decentralized Identifier
ECDSA	Elliptic Curve Digital Signature Algorithm
EVM	Ethereum Virtual Machine
GCM	Galois/Counter Mode
GDPR	General Data Protection Regulation
IPFS	InterPlanetary File System
JWT	JSON Web Token
KEM	Key Encapsulation Mechanism
MCP	Model Context Protocol
MFA	Multi-Factor Authentication
ML	Machine Learning
OAEP	Optimal Asymmetric Encryption Padding
PII	Personally Identifiable Information
RBAC	Role-Based Access Control

RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
SOC	Security Operations Centre
UEBA	User and Entity Behaviour Analytics
UI	User Interface
WSS	WebSocket Secure

POs & PSOs addressed with Justification

Project Title	POs & PSOs addressed	Justification
Aegis Cyber Shield	PO1 – Engineering Knowledge	Applies cryptography (AES, RSA, SHA-256, ECDSA), distributed systems, blockchain and machine learning fundamentals to the problem of secure file sharing.
	PO2 – Problem Analysis	Identifies weaknesses of centralized file sharing such as single points of failure, mutable logs and unscreened uploads, and analyzes them against Zero Trust and compliance requirements.
	PO3 – Design/Development of Solutions	Designs a five-layer architecture with separate trust boundaries for file bytes, integrity references and operational metadata.
	PO4 – Conduct Investigations of Complex Problems	Reviews 30 research papers and standards, defines test cases for integrity, access control and threat screening, and plans measurement of results.
	PO5 – Modern Tool Usage	Uses React, TypeScript, Express.js, FastAPI, Solidity, Hardhat, IPFS (Pinata) and MongoDB, and recognizes their limitations such as IPFS availability and on-chain privacy.
	PO6 – The Engineer and The World	Protects confidential files, supports GDPR and ISO/IEC 27001 aligned controls, and reduces the societal and economic impact of data breaches.
	PO7 – Ethics	Keeps personal data and raw file content off-chain, avoids exaggerated claims about certification, and records user actions transparently.

Project Title	**POs & PSOs addressed**	**Justification**
	PO8 – Individual and Collaborative Team Work	The front end, gateway, AI engine, smart contracts and storage modules are shared among the four team members and integrated together.
	PO9 – Communication	Produces architecture diagrams, data-flow descriptions, a weekly project diary, a report and review presentations.
	PO10 – Project Management and Finance	Follows a seventeen-week plan with weekly milestones, module integration, testing and documentation checkpoints.
	PO11 – Life-long Learning	Works with fast-moving areas such as blockchain, decentralized storage, AI security analytics and post-quantum cryptography.

Project Title	POs & PSOs addressed	Justification
	PSO1	Apply computational structures and algorithms to solve complex problems. Uses hashing, hybrid encryption, entropy analysis, risk scoring and smart-contract logic.
	PSO2	Design, develop and evaluate computer-based systems using advanced technologies. Develops a full-stack, multi-service secure file-sharing system that integrates AI, blockchain and IPFS.
	PSO3	Combines AI threat screening, Zero Trust access control, hybrid encryption, IPFS and blockchain for secure, auditable file sharing. Team-based development and planned testing support continuous learning and collaboration.

Sustainable Development Goals (SDGs)

SDG 1	**No Poverty:** End poverty in all its forms everywhere.
SDG 2	**Zero Hunger:** End hunger, achieve food security and improved nutrition and promote sustainable agriculture.
SDG 3	**Good Health and Well-being:** Ensure healthy lives and promote well-being for all at all ages.
SDG 4	**Quality Education:** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.
SDG 5	**Gender Equality:** Achieve gender equality and empower all women and girls.
SDG 6	**Clean Water and Sanitation:** Ensure availability and sustainable management of water and sanitation for all.
SDG 7	**Affordable and Clean Energy:** Ensure access to affordable, reliable, sustainable and modern energy for all.
SDG 8	**Decent Work and Economic Growth:** Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.
SDG 9	**Industry, Innovation and Infrastructure:** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.
SDG 10	**Reduced Inequalities:** Reduce inequality within and among countries.
SDG 11	**Sustainable Cities and Communities:** Make cities and human settlements inclusive, safe, resilient and sustainable.
SDG 12	**Responsible Consumption and Production:** Ensure sustainable consumption and production patterns.

SDG 13	**Climate Action:** Take urgent action to combat climate change and its impacts.
SDG 14	**Life Below Water:** Conserve and sustainably use the oceans, seas and marine resources for sustainable development.
SDG 15	**Life on Land:** Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, and halt and reverse land degradation and halt biodiversity loss.
SDG 16	**Peace, Justice and Strong Institutions:** Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels.
SDG 17	**Partnerships for the Goals:** Strengthen the means of implementation and revitalize the global partnership for sustainable development.

SDGs addressed with Justification

Project Title	SDG addressed	Justification
Aegis Cyber Shield	SDG 9: Industry, Innovation and Infrastructure	The project integrates Zero Trust access control, AI-based threat screening, hybrid encryption, IPFS and blockchain into one secure digital infrastructure. It contributes to resilient and innovative information infrastructure for organizations.
	SDG 16: Peace, Justice and Strong Institutions	Tamper-evident hash registration, per-operation provenance records and a chain-of-custody trail support accountability, forensic investigation and transparent institutions. Encryption before storage protects the privacy of individuals and organizations.
	SDG 8: Decent Work and Economic Growth	Reducing the exposure of shared business files lowers the risk and cost of data breaches. Automated compliance scoring reduces manual audit effort and lets teams collaborate securely and productively.
	SDG 3: Good Health and Well-being	Medical reports and lab files shared by clinics and research groups often contain personal identifiers. The system flags these before storage and encrypts files to limit exposure. The health link is indirect and not yet tested on real medical records.
	SDG 4: Quality Education	Schools and colleges share course material, assessments and research data, and need to know who accessed each file. Encrypted sharing with expiry, download limits and an audit trail supports this. The design can also serve as teaching material for applied security courses.

Chapter 1

INTRODUCTION

1.1 General

Digital files are routinely shared between individuals, teams, devices and organizations, and the volume of sensitive information that travels through file-sharing services grows every year. Documents, source code, financial records, personal data and research material are all exchanged through web platforms and cloud storage. This convenience has made file sharing one of the most attractive targets for attackers. Escalating cyber threats, ranging from credential theft and ransomware to insider misuse, mean that a file-sharing service must do much more than authenticate a user at login. The cost of failure is also high: the IBM Cost of a Data Breach Report 2023 places the global average cost of a data breach at USD 4.45 million. A file that is shared must therefore be checked, protected and verified throughout its entire life cycle.

Most widely used platforms still follow a centralized design in which a single server, or a single cloud account, stores the files and the database that describes them. Such a design creates a single point of failure and a single trust boundary. If the server or an administrator account is compromised, a large number of files may be exposed together, especially when files are stored in a form that the server itself can read. The upload path of a basic platform may accept malicious or sensitive files without adequate screening, and the platform usually offers the user no independent way to confirm that a downloaded file is identical to the file that was uploaded. Access control is often limited to static roles, which cannot represent context such as an expired share, an unusual session, an unfamiliar location or abnormal behaviour. Audit logs kept inside the same application

database may be edited or incomplete, which weakens incident investigation. Finally, tracking compliance with frameworks such as GDPR, ISO/IEC 27001 and NIST CSF is still largely a manual activity that is difficult to keep consistent.

Recent technologies provide building blocks that address parts of this problem. The Zero Trust model, described in NIST SP 800-207, requires every access request to be verified using identity, device and policy rather than network location. The InterPlanetary File System (IPFS) provides content-addressed storage in which a file is identified by a Content Identifier (CID) derived from its content. Blockchain provides an append-only, tamper-evident ledger that can record selected file hashes and access events for later verification. Artificial intelligence and data-analytics techniques can flag suspicious or sensitive content before it is shared. Individually, these technologies are well established, but each of them solves only one part of the problem. IPFS alone does not give confidentiality or access control, a blockchain alone does not store files, and an AI scanner alone does not protect stored data.

This project, named Aegis Cyber Shield, explores these controls together in one secure file-sharing prototype. Every request is verified by a Zero Trust gateway. Each uploaded file is scanned by an AI security engine, encrypted using a hybrid scheme, stored on IPFS, and registered on a blockchain registry through its hash and CID. Sharing, expiry, download limits and QR-based links are enforced on every download, and each operation produces a provenance record that can be used for forensics and compliance reporting. The project does not claim to invent these technologies; its contribution is the integration of the controls into a traceable upload-to-retrieval workflow together with a plan to evaluate its behaviour under defined test cases. As an academic prototype, it does not claim enterprise-scale security or formal compliance certification.

1.2 Objective of the Work

The primary objective of this project is to design and develop a secure, decentralized and verifiable file-sharing system in which access validation, threat screening, encryption, decentralized storage and tamper-evident activity records operate as one connected workflow rather than as unrelated features. The system aims to reduce the exposure that results from a single central server, to provide users with an independent integrity check,

and to give security teams a traceable record of what happened to each file. The work also aims to build the prototype in such a way that each security claim can be tested and supported by measured evidence.

The specific objectives of this project are as follows:

- To apply Zero Trust, request-level access control by continuously verifying identity, session, role, behaviour and risk for every file operation, including multi-factor authentication when the risk score is high.
- To inspect uploaded files with a multi-stage AI security engine that identifies malware indicators, high-entropy (packed or encrypted) content, sensitive personal data and hidden payloads, and that maps findings to the MITRE ATT&CK framework.
- To protect file confidentiality using hybrid encryption, with AES-256-GCM for file content and RSA-4096-OAEP for wrapping the per-file key, so that the server stores only ciphertext.
- To store encrypted content on IPFS through Pinata, with a local mock fallback, and to identify each stored file with a CID.
- To register file hashes, CIDs, signatures and permissions on a blockchain through Solidity smart contracts, giving an independent integrity reference.
- To support controlled sharing with recipients, expiry, download limits and QR links, and to re-check the policy at every download.
- To maintain a provenance and audit history for upload, share, revoke, download and denied-access events, and to provide a real-time security operations dashboard with automated compliance scoring.
- To evaluate the prototype for functional correctness, integrity, access control, threat-screening behaviour, retrieval time and usability.

1.3 Motivation for this Work

The motivation for this work arises from the gap between the number of security controls that exist and the way they are usually combined in everyday file-sharing tools. A file

passes through several risks in its life: the identity and permission of the user must be checked, the content may contain malware or sensitive information, the stored bytes must remain confidential, a later download must be checked for integrity, and investigators need a traceable activity record. In a typical centralized platform, these concerns are handled by separate components, or are not handled at all. For example, an upload may be scanned by a separate process whose result is not preserved, storage may be readable by anyone who controls the server, and the audit trail may be an ordinary table that can be modified. When these controls do not share information with one another, an attacker or insider can exploit the weak link between them.

A second motivation is the growing demand for verifiable evidence. Regulations such as GDPR Article 32 call for security measures that are appropriate to the risk, including confidentiality, integrity, availability and resilience, and standards such as ISO/IEC 27001 and the NIST Cybersecurity Framework expect logging, cryptography and access control to be demonstrable. Organizations also need to reconstruct incidents, which requires evidence of who performed an action, what happened, when it happened and which file and transaction were involved. At the same time, behavioural and geographic anomalies cannot be detected by static role-based access control alone. Decentralized storage and blockchain records offer a way to keep file content and integrity evidence outside a single administrator's control, while AI-assisted analysis offers a way to flag risk before a file is shared. These observations motivate the study of an integrated design that treats the security controls as part of the file life cycle, and that is evaluated through end-to-end tests rather than through component-level claims.

1.4 Methodology of the Project

The methodology for developing the Aegis Cyber Shield follows a structured, layered and incremental approach. The work begins with requirement analysis, where the risks that a file faces during upload, storage, sharing and download are converted into concrete requirements: identity verification, permission enforcement, content inspection, confidentiality, integrity verification and traceability. A literature survey of research papers, technical standards and official security guidance is carried out to support each design choice, and the survey is grouped into themes such as Zero Trust and governance, de-

centralized storage, blockchain integrity, cryptography, AI threat detection, behavioural analytics and security operations. The outcome of this stage is a five-layer architecture comprising presentation, gateway, intelligence, integrity and storage layers, together with defined trust boundaries that separate encrypted file bytes, integrity references and operational metadata.

In the implementation stage, each layer is developed as an independent module. The presentation layer is a React and TypeScript dashboard. The gateway is an Express.js service that applies a six-stage Zero Trust pipeline to each request. The intelligence layer is a Python FastAPI service that performs malware, entropy, personal-data and steganography scans and evaluates session risk. The integrity layer consists of Solidity contracts, FileRegistry and DecentralizedID, executed on a local Hardhat EVM with an in-memory ledger fallback. The storage layer pins encrypted files to IPFS through Pinata, with a local mock fallback, while MongoDB (or a development mock) stores users, file records and activity records. During an upload, a file is scanned, encrypted with a fresh AES-256-GCM key, the key is wrapped with the RSA public key, the SHA-256 hash is computed and signed with ECDSA, the ciphertext is pinned to IPFS, and the hash and CID are registered on the blockchain.

The download and sharing flows apply the same discipline in reverse. When a user requests a file, the gateway validates identity and permission, checks ownership or the active share policy, confirms access through the on-chain registry, fetches the ciphertext using its CID, verifies the hash, unwraps the AES key and decrypts the file only for the authorized recipient. Sharing supports direct user shares and QR links with a 32-byte random token, expiry and optional download limits, and the policy is re-checked on every download. Each operation generates a FileActivity record with the actor, time, outcome, device and location details, and a transaction reference where applicable. Real-time events are delivered through WebSocket, and nine security tools are exposed to compatible AI clients through the Model Context Protocol.

Finally, the project follows a validation and documentation approach. Test cases are defined for authentication, authorization, file validation, integrity, storage retrieval, audit logging, performance and usability, and the input, expected outcome and observed result of every test case is recorded. Module progress is estimated provisionally and is to be validated against code, logs and demonstrable tests. The work is carried out over a

seventeen-week plan, in which Weeks 1 to 10 cover research, design and initial module development, and Weeks 11 to 17 cover integration, negative testing, evidence collection and documentation for the Phase 1 review.

Chapter 2

LITERATURE REVIEW

2.1 LITERATURE SURVEY

The literature survey covers 30 sources, comprising 16 research papers and reports and 14 standards and official documents. The sources are grouped into themes: Zero Trust and governance, decentralized storage, blockchain integrity, cryptography, AI threat detection, user and entity behaviour analytics, security operations and forensics, and supporting tools. The most relevant sources are summarized below, and the supporting standards are summarized in Section 2.2.

2.1.1 Zero Trust Architecture

- **Authors:** S. Rose, O. Borchert, S. Mitchell, S. Connelly
- **Source:** NIST Special Publication 800-207, 2020

This publication defines Zero Trust Architecture (ZTA) as a security approach in which no user, device or request is trusted solely because of its network location. Instead, every request to access a resource is evaluated using the identity of the subject, the state of the device, the sensitivity of the resource and a defined access policy. The document describes logical components such as a policy engine, a policy administrator and a policy enforcement point, and discusses deployment variations and threats to the architecture.

The central finding is that access decisions must be made continuously and per request, rather than once at the network perimeter. Perimeter-based defences are weak against advanced persistent threats and lateral movement, because an attacker who has

entered the network inherits the trust that the perimeter grants. By moving enforcement close to each resource and re-evaluating context, a Zero Trust design limits what a single compromised account or host can reach.

In this project, the publication motivates the Express.js gateway, which acts as a policy enforcement point. The gateway checks the token, source address, session state, role and attribute policy, behavioural score and risk for each request, and the same policy is re-checked on every download.

2.1.2 IPFS – Content Addressed, Versioned, P2P File System

- **Authors:** J. Benet
- **Source:** arXiv:1407.3561, 2014

This paper presents the InterPlanetary File System, a peer-to-peer distributed file system that connects computing devices with the same system of files. Files are broken into blocks that are identified by a cryptographic hash, and the blocks are linked together into a Merkle directed acyclic graph. The resulting Content Identifier (CID) depends on the content of the file and not on its location, which gives deduplication and makes any modification of the content detectable.

The paper also describes content-addressed retrieval through a distributed hash table and block exchange between peers. An important practical observation is that a CID guarantees what the content is, but it does not by itself guarantee that the content remains available or that it is confidential. Availability depends on nodes that keep the data, which is why pinning services are used, and anyone who knows a CID can retrieve unencrypted content.

For this project, IPFS is used to hold the encrypted file only. The file is encrypted before it is pinned, the CID is recorded on the blockchain registry, and the pinning service (Pinata, with a local mock fallback) is used to keep the content available.

2.1.3 How to Time-Stamp a Digital Document

- **Authors:** S. Haber, W. S. Stornetta
- **Source:** Journal of Cryptology, 3(2), 1991

This early paper addresses the problem of proving that a digital document existed at a certain time and has not been altered since. The authors propose time-stamping schemes in which the hash of a document is linked to previously issued time-stamps, so that creating a back-dated or altered record would require recomputing every later link. The scheme does not require the content of the document to be revealed to the time-stamping service, because only its hash is submitted.

The key idea is that a small, linked and publicly verifiable record can provide strong evidence about a much larger document. This idea underlies later hash-linked ledgers, including blockchains such as Bitcoin, which make past tampering detectable.

The project follows the same principle. The system anchors the SHA-256 hash and CID of each encrypted file on the blockchain registry and not the file itself. This limits cost, keeps the file content private and provides an independent reference against which a later download can be compared.

2.1.4 Blockchain-Enabled Intrusion Detection and Prevention System of APTs within Zero Trust Architecture

- **Authors:** C. Alevizos et al.
- **Source:** IEEE Access, 10, 89270–89288, 2022

This paper proposes a system that combines a Zero Trust architecture with blockchain technology to detect and prevent advanced persistent threat (APT) intrusions. The work argues that APTs are difficult to defend against with perimeter security alone, and that the shared, tamper-evident state of a blockchain can support trustworthy detection and response across the components of a Zero Trust deployment.

The paper is the closest prior work to the present project, because it also uses Zero Trust and blockchain together for security. Its focus, however, is intrusion detection and prevention for APTs and not the complete life cycle of a shared file. The remaining gap, which the present project addresses, is a full file-sharing workflow in which screening, encryption, content addressing, on-chain registration, access checks and provenance are bound together.

2.1.5 Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains

- **Authors:** E. Androulaki et al.
- **Source:** EuroSys, 2018

This paper describes Hyperledger Fabric, a modular blockchain platform designed for permissioned enterprise settings. Unlike public blockchains in which transactions are ordered and then executed by every node, Fabric uses an execute-order-validate architecture. Chaincode is first executed by endorsing peers, the resulting transactions are ordered by an ordering service, and each peer then validates them before committing them to the ledger.

The paper highlights that a permissioned ledger gives verifiable shared state and audit trails to a known group of participants, while offering better control of confidentiality and performance than an open network. It also illustrates the design trade-offs between decentralization, throughput and privacy.

This work motivates the audit-trail design of the project and the decision to keep the on-chain records minimal. In the prototype, Solidity contracts run on a local Hardhat EVM, but the same registry concept could later be moved to a permissioned ledger.

2.1.6 Applications of Blockchains in the Internet of Things: A Comprehensive Survey

- **Authors:** M. S. Ali et al.
- **Source:** IEEE Communications Surveys & Tutorials, 2019

This survey reviews how blockchain technology is applied in the Internet of Things and discusses the open challenges that accompany these applications, including scalability, storage, privacy and transaction cost. The authors note that storing large volumes of data directly on a blockchain is impractical, because every full node must replicate the data and the entries are difficult to remove once published.

The relevance of the survey to this project is the design implication that only compact integrity references should be placed on the chain. The project therefore keeps file

contents encrypted on IPFS and registers only file hashes, CIDs, threat scores, signatures and permissions on the blockchain, which reduces cost and limits privacy exposure.

2.1.7 The Role of Machine Learning in Cybersecurity

- **Authors:** G. Apruzzese et al.
- **Source:** ACM Digital Threats: Research and Practice, 4(1), 2023

This article reviews how machine learning is used in cybersecurity and examines the practical limitations that arise when such systems are deployed. It discusses issues such as data quality, evaluation methodology, adversarial behaviour and the cost of false alarms, and it argues that machine-learning detectors should not be treated as infallible.

The authors' main message is that learned detectors are most useful when they are combined with other controls and are evaluated honestly. For this project, the finding supports a hybrid design in which rule-based checks and analytics are combined, in which every AI finding is recorded with its evidence, and in which false positives and negatives are measured against test data rather than ignored.

2.1.8 A Mathematical Theory of Communication

- **Authors:** C. E. Shannon
- **Source:** Bell System Technical Journal, 27(3), 1948

Shannon's paper founded information theory and defined entropy as a measure of the information content, or unpredictability, of a source. Data whose bytes are close to uniformly random has high entropy, whereas ordinary text and many structured formats have lower entropy.

This property is used in malware and ransomware analysis. Packed executables and encrypted content have unusually high byte entropy, so a high entropy score can indicate that a file is packed, encrypted or obfuscated. In the project, an entropy analyzer inside the AI security engine uses this idea to flag packed, ransomware-like files, and the result is treated as a risk indicator and not as a final verdict.

2.1.9 Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization

- **Authors:** I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani
- **Source:** ICISSP, 2018

This paper introduces the CICIDS2017 dataset, which contains benign traffic and a range of common attacks generated in a controlled testbed, together with an analysis of traffic features and a characterization of the attacks. The dataset was created to overcome shortcomings of older intrusion-detection datasets, such as outdated attack types and limited diversity.

In the project, the dataset is considered as a candidate benchmark for evaluating the detection and behaviour-analysis modules. The planned evaluation of the AI engine on benchmark data is part of the future work, because detection quality depends on data and must be demonstrated by measurement.

2.1.10 Keystroke Dynamics as a Biometric for Authentication

- **Authors:** F. Monrose, A. D. Rubin
- **Source:** Future Generation Computer Systems, 2000

This paper studies keystroke timing as a behavioural biometric. The way a person types, including the duration for which keys are held and the intervals between them, forms a pattern that can be compared with a stored profile to support user authentication. The authors evaluate classification methods on typing data and show that such patterns can provide useful identity evidence.

A related line of work by T. Bass proposes that fusing data from several sensors improves intrusion detection. Together, these works motivate the behavioural part of the gateway. In the project, keystroke dynamics and mouse jitter arrive as HTTP headers, and they are combined with device and geographic signals into one risk score between 0 and 100 per session. Multi-factor authentication is triggered when the score reaches 70 or more.

2.1.11 Security Operations Center: A Systematic Study and Open Challenges

- **Authors:** M. Vielberth et al.
- **Source:** IEEE Access, 8, 227756–227779, 2020

This paper presents a systematic study of security operations centres (SOCs), covering their people, processes and technology, and it identifies open challenges such as alert overload, integration of tools and lack of automation. The study shows that effective monitoring requires the integration of data from several sources into a coherent view.

This motivates the live dashboard, alerts and incident workflow of the project. File-sharing events, threat alerts and compliance information are presented to a SOC analyst in one place, and a set of cooperating agents (scanner, responder, forensics and compliance) automates routine analysis.

2.1.12 Getting Physical with the Digital Investigation Process

- **Authors:** B. Carrier, E. H. Spafford
- **Source:** CERIAS Technical Report 2003-29, 2003

This report proposes a process model for digital investigation that is based on the investigation of physical crime scenes. It frames digital activity as evidence that must be preserved, interpreted and documented through a defined process, so that conclusions can be defended.

For the project, this supports the design of the per-operation FileActivity record. Each action stores the event type, actor, IP address, geolocation, device fingerprint, recipient, expiry, transaction hash and outcome, so that a chain of custody can be reconstructed for any file.

2.2 Standards and Official Guidance Reviewed

In addition to the research papers, the following standards and official documents were reviewed. Each of them is mapped to a feature of the system.

- **NIST FIPS 197 (AES) and NIST SP 800-38D (GCM):** AES is a standard block cipher with 128-, 192- and 256-bit keys, and Galois/Counter Mode provides confidentiality together with an authentication tag. The system uses AES-256-GCM for each file, so that tampering with the ciphertext is detected on download.
- **RFC 8017 (PKCS #1 v2.2) and the Diffie–Hellman and RSA papers:** public-key cryptography lets parties share keys without a prior secret channel, and RSA-OAEP provides a secure padding scheme. The system uses RSA-4096-OAEP to wrap the per-file AES key for the recipient.
- **NIST FIPS 180-4 (SHA-2) and FIPS 186-5 (DSS):** SHA-256 digests are used to verify files against stored hashes, and ECDSA signs each file hash with the user's key.
- **NIST FIPS 203 (ML-KEM):** a lattice-based key-encapsulation standard derived from CRYSTALS-Kyber. The project simulates a Kyber-1024 KEM for post-quantum readiness only; it is not deployed.
- **NIST CSF 2.0, ISO/IEC 27001:2022 and GDPR Article 32:** these define outcomes and controls for risk management, logging (control 8.15), cryptography (control 8.24) and security of processing. The compliance scorecards of the system map its controls to these frameworks.
- **MITRE ATT&CK:** a catalogue of adversary tactics and techniques. Each alert of the AI engine is mapped to a tactic and technique identifier.
- **OWASP Top 10:2021:** identifies broken access control and cryptographic failures as leading web risks. The gateway therefore enforces server-side access checks and encryption by default.
- **Model Context Protocol specification and IBM Cost of a Data Breach Report 2023:** the first defines a standard way for AI clients to call external tools and is used to expose nine security tools, and the second provides the average breach cost figure of USD 4.45 million used for motivation.

2.3 Research Gap

The reviewed literature shows that the individual building blocks are well established. Zero Trust, IPFS, blockchain, hybrid cryptography and machine-learning-based detection have each been studied in depth. The comparison in Table 2.1 is qualitative and is given only for discussion. It shows that existing approaches address only some of the required capabilities, and that prior work such as Alevizos et al. already combines Zero Trust and blockchain for intrusion detection. The remaining gap is integration at the level of the complete file-sharing workflow.

Capability	**Cloud sharing**	**IPFS only**	**Chain only**	**Aegis**
Single point of failure	Yes	No	No	Partly
Content-addressed storage	No	Yes	No	Yes
Tamper-evident record	Varies	Partial	Yes	Yes
Encryption before storage	Varies	No	No	Yes
Behaviour checks at access	Rare	No	No	Yes
Compliance monitoring	Varies	No	No	Yes

Table 2.1: Integration gap compared with existing approaches (qualitative)

The gap addressed by this project is a single workflow that binds AI screening, encryption, content addressing, on-chain registration, access checks and provenance for a file-sharing service that is visible to a security operations team. The literature also highlights practical limitations that the project must address: a CID does not guarantee availability or confidentiality, on-chain data is difficult to remove, AI detection depends on data quality and produces false positives and negatives, and logs alone do not prove that every control is correctly enforced. These limitations are carried into the risk analysis and the testing plan.

Chapter 3

SYSTEM DESIGN

System design is the process of designing the elements of a system such as the architecture, modules and components, the different interfaces of those components and the data that flows through that system. The design of Aegis Cyber Shield separates user interaction, access enforcement, threat analysis, integrity records and file storage into five layers, so that each layer has a clear responsibility and a clear hand-off to the next.

3.1 System Architecture

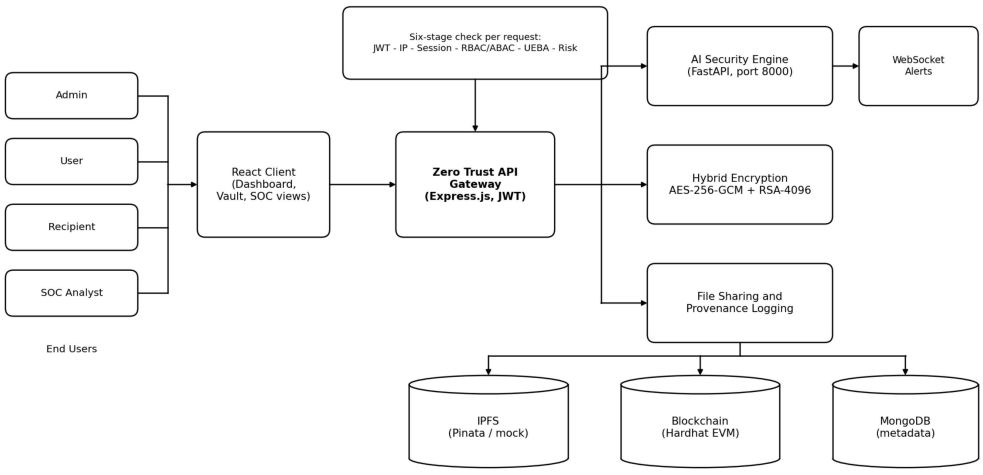


Figure 3.1: System Architecture of Aegis Cyber Shield

The architecture has five layers. The Presentation layer is a React and TypeScript dashboard with more than fifteen pages (development port 3000). The Gateway layer is

an Express.js Zero Trust API gateway (port 5000). The Intelligence layer is a FastAPI service for multi-stage scanning and behaviour analysis (port 8000). The Integrity layer is an EVM smart-contract registry with decentralized identity management (port 8545). The Storage layer is IPFS content-addressed storage through Pinata with a mock fallback. Supporting services include a MongoDB metadata store (port 27017) with a mock fallback, WebSocket event notifications, hash and audit records, and test scripts and logs. The ports quoted are development defaults.

Layer	**Main responsibility**	**Output / hand-off**
Presentation	Collect user actions and show status	File request, scan result and sharing state
Gateway	Authenticate, authorize and validate requests	Approved request or a recorded denial
Intelligence	Inspect file content and calculate risk indicators	Scan outcome and explanation
Integrity	Register and verify hash and CID references	Transaction or registry result
Storage	Keep encrypted content addressable	CID used for retrieval

Table 3.1: Responsibility of each layer

3.1.1 End Users and Actors

The system is designed around six actors, each with defined privileges. The **User** registers, logs in, uploads, downloads and shares files. The **Recipient** receives files that are shared with them through the shared inbox. The **Public** actor is an unauthenticated person who opens a QR share link, and can access only the single file that the link represents, subject to expiry and download limits. The **Admin** manages users, roles, IP allow-lists and system health. The **SOC analyst** monitors alerts, the threat map, forensic timelines and compliance scorecards. The **AI agent** accesses nine security tools through the Model Context Protocol. A role-based model, together with attribute-based rules, assigns each actor only the features and data that are relevant to their responsibilities. For example, Auditor and Guest roles are limited to business hours.

3.1.2 Presentation Layer

The Presentation layer is the interface through which users interact with the system. It is developed with React 18, TypeScript and Tailwind CSS, and offers pages for registration and login, the encrypted file vault, upload with scan results, sharing and QR links, the shared inbox, the SOC dashboard, the threat map, analytics, file life cycle, forensics, compliance scorecards and an AI copilot. Toast notifications replace blocking alert dialogs, and the shared inbox shows Active, Expired and Limited badges so that the user can understand the sharing state at a glance. The layer collects user actions and shows the status of scans, uploads and shares, but it does not make security decisions; all enforcement is performed on the server side.

3.1.3 Zero Trust Gateway

The Zero Trust Gateway is the core access-control component. Every request to a protected endpoint passes through a pipeline of six checks, shown in Figure 3.2. JWT access tokens last two hours, and refresh tokens last seven days and rotate. Per-user IP allow-lists are supported, with a bypass for private addresses, and concurrent sessions are detected. The role-based and attribute-based policy engine limits Auditor and Guest roles to business hours. Keystroke dynamics and mouse jitter arrive as HTTP headers and are scored by the behaviour evaluator, and an AI risk assessment produces a combined 0 to 100 risk score per session. When the score reaches 70 or more, multi-factor authentication is triggered. A request that fails any stage is denied, and the denial is recorded in the audit log.

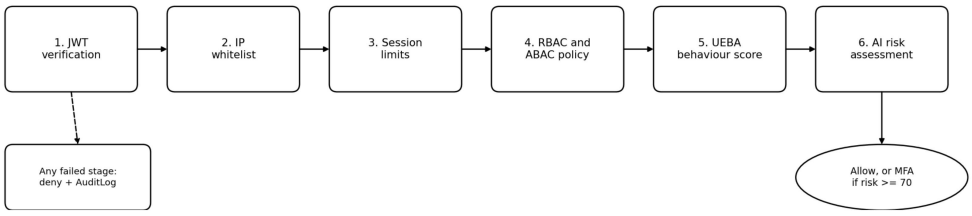


Figure 3.2: Zero Trust Gateway Pipeline

3.1.4 AI Security Engine

The AI Security Engine is a Python 3.9+ FastAPI service that inspects each file before it enters the normal sharing path. It combines rule-based heuristics with analytics, and every finding is returned with an explanation, so that results can be traced and false positives can be studied. The engine contains five analyzers. The **malware scanner** applies heuristic detection. The **entropy analyzer** flags packed or ransomware-like files with unusually high byte entropy. The **personal-data detector** searches for sensitive identifiers such as social security numbers, card numbers, Aadhaar numbers and passport numbers. The **steganography scanner** looks for hidden archives inside JPEG or PNG images. The **UEBA evaluator** calculates continuous session risk. Each alert is mapped to a MITRE ATT&CK tactic and technique. The response to a flagged file, whether rejection, quarantine or manual review, is a policy choice that is implemented and tested in the prototype.

3.1.5 Hybrid Encryption Module

The Hybrid Encryption Module protects file confidentiality and integrity. Encryption is carried out with the Node.js crypto library. A fresh AES-256-GCM key is generated for each file, and GCM supplies an authentication tag so that tampering with the ciphertext is detected on download. The AES key is then wrapped with the recipient's RSA-4096 public key using OAEP padding, so that a stored wrapped key cannot decrypt a file without the matching RSA private key. A SHA-256 hash of the file is computed and signed with ECDSA using the user's key. On the download path, the ciphertext is retrieved, the hash is verified, the AES key is unwrapped, and the file is decrypted. A Kyber-1024 key-encapsulation mechanism is simulated to demonstrate post-quantum readiness, but it is not deployed. Key handling follows the principle that cryptographic strength depends on the generation, protection, storage and life cycle of keys, and no private key or raw file content is placed in public blockchain records.

3.1.6 Blockchain Integrity Layer

The Integrity layer provides an independent reference for the integrity of each file. Two Solidity contracts are developed with Hardhat and ethers.js. `FileRegistry.sol` pro-

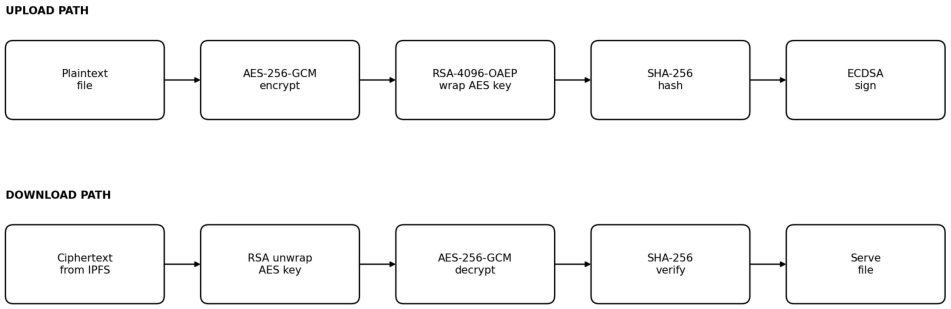


Figure 3.3: Hybrid Encryption: Upload and Download Paths

vides the functions registerFile, grantAccess, revokeAccess, verifyAccess and setEmergencyLock. `DecentralizedID.sol` provides registerDID and getDID for decentralized identities. The contracts execute on a local Hardhat EVM, and an in-memory ledger is used as a fallback when the EVM is not available. The chain stores file hashes, CIDs, threat scores, signatures and permissions. File contents stay encrypted on IPFS. Because entries on a blockchain are difficult to remove after publication, the records are kept minimal and do not contain personal information.

3.1.7 Decentralized Storage Layer

The Storage layer keeps the encrypted file addressable through a CID. Encrypted files are pinned to IPFS using the Pinata service so that they remain available, and a local mock fallback allows development and demonstration when the service is not reachable. Because a CID guarantees content identity but not availability, retrieval from the pinned CID is part of the test plan. The ciphertext is the only data placed in this layer.

3.1.8 Database and Data Models

MongoDB (or a development mock) stores the operational metadata that is not suitable for the blockchain. Four main data models are used. The **User** record contains the username, bcrypt password hash, email, role, DID address, RSA public key, encrypted RSA private key, device fingerprint base, biometric cadence, whitelisted IPs, refresh tokens and an active flag. The **FileDocument** contains the file name, size, MIME type, CID, SHA-256 hash, owner, threat score, encrypted AES key, initialization vector, digital sig-

nature, blockchain transaction hash, the sharedWith list, and provenance and version fields. The **FileActivity** record stores the file reference, event type, actor, IP address, geolocation, success or failure reason, transaction hash and metadata. The **ThreatIntelligence** record stores the event type, severity, source and target address and location, threat score, confidence, MITRE tactic and technique, description, active state and tags. Each component stores only the information it needs.

3.1.9 File Sharing Module

The sharing module supports two mechanisms. In *share with user*, the owner sends a POST /share request with the target, duration and maximum downloads. The gateway validates the owner, finds the recipient, adds a share entry with expiry and limits, logs the activity and notifies the recipient through WebSocket. In *QR link sharing*, the owner calls POST /generate-share-link, a 32-byte random token is created and stored in the sharedWith list, and the link is rendered as a QR code. The recipient opens /share/:token, and the public-download endpoint checks expiry and limits before the file is served, without requiring a login. The policy is re-checked at every download, so that expired or revoked shares are denied.

3.1.10 Provenance and Audit Module

Every operation on a file produces a FileActivity record. The record contains the event type, actor identity, IP address and geolocation (city, country, latitude, longitude and ISP), device fingerprint, recipient, expiry, blockchain transaction hash and whether the operation succeeded or failed. Downloaded PDF files are watermarked to support evidence preservation. These records support chain of custody and give investigators a clear upload, share and download timeline for each file. The audit log can additionally be anchored on the blockchain, so that it cannot be silently edited.

3.1.11 Agent Swarm, Threat Oracle and MCP Tools

Four software agents run in parallel. The **scanner** agent looks for threats, brute-force attempts and geographic anomalies. The **responder** agent handles incident response, including an emergency lockdown. The **forensics** agent verifies chain of custody and

blockchain integrity. The **compliance** agent evaluates GDPR Article 32, ISO/IEC 27001 and NIST CSF. A **threat oracle** produces a 72-hour forecast over eight threat patterns: ransomware, DDoS, phishing, exfiltration, brute force, zero-day, supply chain and insider. In addition, nine security tools are exposed over JSON-RPC and server-sent events through the Model Context Protocol, so that compatible AI clients can query the system.

3.1.12 SOC Dashboard and Compliance Monitoring

The SOC dashboard gives a security analyst a live view of alerts, threat maps, activity feeds and scorecards, delivered through WebSocket events. The compliance engine maps implemented controls to GDPR Article 32, ISO/IEC 27001:2022 and NIST CSF 2.0 outcomes, and produces automated scores. These scores describe the behaviour of the prototype against selected controls; they do not represent a formal certification.

3.2 Use Cases

The use cases identified for the system are listed in Table 3.2. The complete use-case list is given in the project documentation.

Use case	**Description**	**Main actor**
UC1–UC2	Register and login	User
UC3	Upload and scan a file	User
UC4	Download and verify a file	User, Recipient
UC5–UC8	Share with a user, set expiry and limits, generate and scan a QR link	User, Recipient, Public
UC9–UC10	SOC dashboard and threat map	SOC analyst
UC11–UC12	Agent swarm and threat oracle	SOC analyst
UC13–UC15	Analytics, file life cycle and copilot	User, SOC analyst
UC16	Manage users and roles	Admin
UC17–UC18	Compliance report and forensics	Admin, SOC analyst
UC19	AI agent analysis through MCP	AI agent

Table 3.2: Use-case summary (UC1 to UC19)

3.3 Security Controls and Compliance Mapping

Table 3.3 maps each implemented control to the standard that it supports.

Control	**Implementation**	**Standard**
Encryption at rest	AES-256-GCM per file	GDPR Art. 32
Key management	RSA-4096-OAEP key wrapping	ISO/IEC 27001:2022 A.8.24
Identity verification	JWT and behavioural biometrics	NIST CSF 2.0 PR.AA
Access control	RBAC, ABAC and Zero Trust	NIST CSF 2.0 PR.AA
Audit logging	AuditLog with blockchain anchoring	ISO/IEC 27001:2022 A.8.15
Incident detection	AI scanning and UEBA	NIST CSF 2.0 DE.AE
Data integrity	SHA-256 and on-chain hash	NIST CSF 2.0 PR.DS
Evidence preservation	Watermarking and custody trail	NIST CSF 2.0 RS.AN

Table 3.3: Security controls and compliance mapping

3.4 API Surface

The gateway exposes REST endpoints grouped by area, as shown in Table 3.4.

Area	Endpoints
Authentication	POST /register, /login, /refresh, /logout, /wallet, /issue-vc, /verify-vc
Files	GET /list; POST /upload; GET /download/:id; POST /share; GET /shared-with-me; POST /generate-share-link; GET /public-share/:token; GET /public-download/:token
Intelligence	GET /analytics/*, /threats/*, /forensics/timeline, /compliance/report; POST /copilot/query
Admin	GET /system-health, /users; POST /update-role, /update-ip
MCP	GET /sse; POST /messages

Table 3.4: API surface of the gateway

Chapter 4

METHODOLOGY

4.1 Requirement Analysis

The development begins with a requirement analysis phase in which the risks that a file faces during its life are converted into concrete requirements. Five risks are identified: the identity and permission of the user must be checked; the content may contain malware or sensitive information; the stored bytes must remain confidential; a later download must be checked for integrity; and investigators need a traceable activity record. These risks lead to six requirements, which are summarized in Table 4.1. Functional requirements include registration and login, upload, download, sharing, revocation, monitoring and reporting. Non-functional requirements include security, usability, performance, availability of stored content and maintainability.

The design principle that follows from this analysis is that security controls should connect to the file life cycle and should not operate as unrelated features. A feasibility study confirms that the required technologies (React, Express.js, FastAPI, Solidity, Hardhat, IPFS and MongoDB) are available as open-source or freely accessible services, and that stable local fallbacks exist for IPFS, the blockchain and the database for development and demonstration.

4.2 System Design

The proposed system adopts a layered modular architecture, described in Chapter 3. Presentation, Gateway, Intelligence, Integrity and Storage are developed as separate

Requirement	Expected system response
Identity verification	Authenticate the user before protected file operations.
Permission enforcement	Check ownership, recipient permissions, link expiry and download limits.
Content inspection	Run configured file checks before the file enters the normal sharing path.
Confidentiality	Encrypt file bytes before external storage.
Integrity verification	Compare the retrieved content hash with a trusted recorded hash.
Traceability	Record upload, share, revoke, download and denied-access events.

Table 4.1: Requirements derived from the problem

modules that communicate through well-defined interfaces. The design separates three kinds of data according to their different storage needs: encrypted file bytes are stored on IPFS and identified by a CID, the file hash and chosen metadata are stored in the blockchain registry, and users, permissions and activity are stored in the application database. Blockchain records are kept minimal, because entries are difficult to remove after publication, and access permissions and encryption keys are never exposed through public metadata. Audit evidence refers to an event and its outcome and does not unnecessarily duplicate private content.

4.3 Zero Trust Access Control

Access control is implemented as a six-stage pipeline in the gateway. First, the JWT is verified. Second, the source address is compared with the per-user allow-list. Third, session limits and concurrent sessions are checked. Fourth, the role-based and attribute-based policy is evaluated, including business-hour restrictions for Auditor and Guest roles. Fifth, the behaviour score is computed from keystroke dynamics, mouse jitter, device and geographic signals. Sixth, an AI risk assessment combines these signals into a score from 0 to 100, and multi-factor authentication is triggered at 70 or above.

The pipeline is applied to every request, including downloads, and not only to login. This follows the Zero Trust principle that trust is never granted because of a previous successful request or network location. The outcome of each decision, whether allowed

or denied, is recorded in the audit log, so that the behaviour of the access-control layer can itself be inspected and tested. Because behavioural scoring can produce false positives, the design treats the score as an indicator that raises the level of verification, and not as an infallible decision.

4.4 AI-Based File Screening

File screening takes place in the upload path, before the file is encrypted and stored. The uploaded file is first validated for type, size and name in line with the OWASP guidance on file uploads. It is then analyzed by the malware scanner, the entropy analyzer, the personal-data detector and the steganography scanner of the AI security engine. Each analyzer returns an indicator together with the evidence that produced it, and the indicators are combined into a threat score. Findings are mapped to MITRE ATT&CK tactics and techniques, which gives a consistent vocabulary for alerts.

The response to a flagged file is governed by policy. A file that passes the configured checks continues to normal encrypted storage, while a flagged file may be rejected, quarantined or sent for manual review. The scan result and the actor and time information are stored for later review. The methodology treats AI findings as risk indicators that must be tested with safe, controlled samples, and the limitations of detection, including false positives and false negatives, are reported openly.

4.5 Hybrid Encryption and Integrity Verification

The encryption methodology follows the hybrid pattern in which a symmetric cipher protects the bulk data and an asymmetric cipher protects the symmetric key. On upload, a unique AES-256-GCM key and initialization vector are generated for the file, and the file is encrypted before it leaves the server. The AES key is wrapped with RSA-4096-OAEP, the SHA-256 hash is computed, and the hash is signed using ECDSA. The wrapped key, initialization vector and signature are stored in the file record, and the hash is registered on the blockchain.

On download, the system fetches the ciphertext by CID, recomputes and compares the SHA-256 hash with the trusted reference, unwraps the AES key using the recipi-

ent's private key and decrypts the file. The GCM authentication tag provides a second, independent check that the ciphertext has not been modified. If either check fails, the download is refused and the failure is logged. A stored wrapped key cannot decrypt a file without the recipient's RSA private key, which is not stored in a form the server can use.

4.6 Decentralized Storage and Blockchain Registration

After encryption, the ciphertext is pinned to IPFS through Pinata, and the returned CID is stored. The system then registers the file hash, CID, threat score and signature in the FileRegistry contract, and the returned transaction hash is stored in the file record and in the activity log. Permissions are granted and revoked through the same contract, and verifyAccess is called during download. An emergency lock can be set to freeze access to a file. Hardhat provides a local EVM for development and testing, and an in-memory ledger fallback keeps the workflow demonstrable when the EVM is not available. Smart-contract access conditions are tested locally, and no claim of a production audit is made.

4.7 System Integration and Automation

In this phase, the modules are integrated into one end-to-end workflow. The React client calls the Express gateway over HTTPS and WebSocket Secure, the gateway calls the FastAPI engine for scanning and risk assessment, the encryption routines prepare the file for IPFS, and ethers.js is used to call the smart contracts. The gateway also writes metadata and activity records to MongoDB. Real-time events are delivered to the dashboard through WebSocket, and the Model Context Protocol server exposes nine tools to AI clients.

Integration complexity is one of the main risks of the project, so small end-to-end test cases and stable mock fallbacks are used to isolate faults. Each interface is tested first with simple inputs, and the complete chain is then verified by an integration test. Automation covers the whole path from upload to registration, and the agent swarm runs periodic scanning, forensics and compliance checks without manual intervention.

4.8 Validation and Testing

The validation plan defines test cases for each security claim, and for each test the input, expected outcome and observed result are recorded. Authentication is tested with valid and invalid login and token scenarios. Authorization is tested by confirming that a non-owner is denied and that an expired or revoked share is denied. File validation uses an unsupported type, an oversized file and a suspicious but safe test sample. Integrity is tested by confirming that the recomputed hash of an unchanged file matches the registered reference, and that a modified file does not. Storage is tested by retrieving the CID while the content is pinned. The audit trail is tested by performing upload, share, revoke, download and failed-access actions and confirming that each event is recorded. Usability is checked by asking whether a user can understand the scan outcome and sharing status.

Table 4.2 lists what will be measured. Safe, controlled test files and labelled samples are used for threat screening, and no harmful content is used.

Measure	**How to test**	**Evidence to record**
Upload correctness	Upload allowed sample files	Success or failure, file size and timestamp
Integrity	Compare original and retrieved hashes	Expected and actual hash match
Access control	Try owner, authorized recipient and unauthorized user	Allow or deny outcome for each case
Threat screening	Use safe, controlled test files or labelled samples	Detection outcome and reason
Retrieval time	Repeat download for files of known sizes	File size and elapsed time
Audit trail	Perform upload, share, revoke and download actions	Event record, actor, time and outcome

Table 4.2: Evaluation measures

4.9 Performance Evaluation and Optimization

Performance evaluation records the upload and download duration against the size of each test file, as well as the time taken by individual stages such as scanning, encryption,

IPFS pinning and blockchain registration. These measurements identify which stage dominates the delay and whether it is acceptable for interactive use. Retrieval time is measured by repeated downloads of files of known size.

Possible optimizations include streaming encryption for large files, asynchronous registration of hashes so that the user is not kept waiting for a transaction, caching of access-check results for short periods, and database indexing for activity queries. The performance of a local development network does not represent the performance of a public blockchain, and this limitation is stated when results are reported.

4.10 Project Schedule

The work is organized as a weekly plan. Weeks 1 to 10 were recorded in the project diary, and Weeks 11 to 17 form the work plan up to the Phase 1 review on Friday, 23 October 2026. The plan is summarized in Table 4.3.

Week	Dates	Activity
1	1–5 Jul	Researching the topic
2	6–12 Jul	Idea confirmation
3	13–19 Jul	Requirement analysis and Zero Trust framework
4	20–26 Jul	Five-layer architecture and data-flow design
5	27 Jul – 2 Aug	Dashboard, API planning and database setup
6	3–9 Aug	AI engine integration and risk assessment
7	10–16 Aug	Blockchain integrity and IPFS storage design
8	17–23 Aug	System module integration and provenance
9	24–30 Aug	Module-level testing and security compliance
10	31 Aug – 6 Sep	Overall review and documentation consolidation
11	7–13 Sep	Continue module integration and resolve interface issues
12	14–20 Sep	Validate the upload, storage and hash-registration path
13	21–27 Sep	Check access control, sharing permissions and expiry rules
14	28 Sep – 4 Oct	Run negative tests and review error handling
15	5–11 Oct	Prepare a repeatable demonstration and collect test evidence
16	12–18 Oct	Consolidate report, diagrams and presentation material
17	19–23 Oct	Confirm implemented modules and prepare for the Phase 1 review

Table 4.3: Weekly project schedule (Weeks 1 to 17)

Chapter 5

DATA FLOW

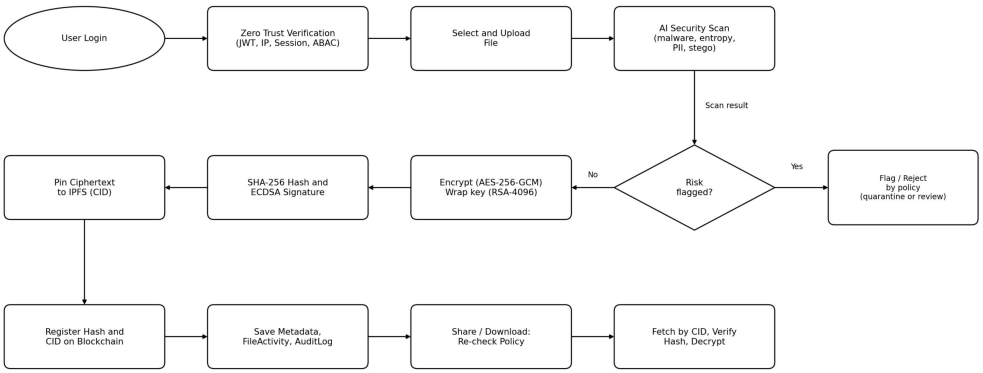


Figure 5.1: Data Flow Diagram

5.1 Description

The flowchart illustrates the complete workflow of the system, describing how a user request is transformed into a protected, registered and verifiable file. The process begins with user login, where the user accesses the system using secure credentials. This is followed by Zero Trust verification, in which the JWT, source address, session, role and attribute policy and behavioural risk are evaluated for the request. This initial stage plays a crucial role in maintaining security, privacy and controlled interaction.

After successful verification, the user selects and uploads a file. The file is passed to the AI security engine, which performs malware, entropy, personal-data and steganogra-

phy scans and returns a threat score with an explanation. If the configured policy flags the file, the file is rejected, quarantined or sent for manual review, and the event is recorded. Only a file that passes the policy continues along the normal storage path, as shown by the decision step in the diagram.

For an accepted file, the system generates a unique AES-256-GCM key, encrypts the file, and wraps the key with RSA-4096-OAEP. The SHA-256 hash of the file is computed and signed using ECDSA. The ciphertext is then pinned to IPFS, which returns the CID, and the hash and CID are registered on the blockchain registry. Finally, the file metadata, the FileActivity record and the AuditLog entry are saved in the database, and the user receives a confirmation.

When the file is later shared or downloaded, the policy is re-checked on every request. The system validates ownership or the active share, confirms access through the registry, fetches the ciphertext by its CID, verifies the hash, and decrypts the file only for the authorized recipient. Success or failure is recorded in the activity log in both cases.

Overall, this workflow represents a layered, verifiable approach to file sharing. Encryption is performed before external storage, file contents are never placed on the blockchain, and enough identifiers are recorded to compare a fetched file with its trusted hash.

5.2 File Upload Sequence

The upload sequence is shown in Figure 5.2. The user sends a POST request to the gateway, which performs the Zero Trust checks and calls the AI engine. After the scan returns a threat score, the gateway encrypts, wraps, hashes and signs the file, pins the ciphertext to IPFS, registers the hash and CID on the blockchain, and saves the FileDocument, FileActivity and AuditLog records in MongoDB before returning a 201 Created response. The scan result and the actor and time information are stored for later review.

5.3 File Download Sequence

The download sequence is shown in Figure 5.3. The user requests a file with GET /download/:id. The gateway applies the Zero Trust checks and verifies ownership or the active

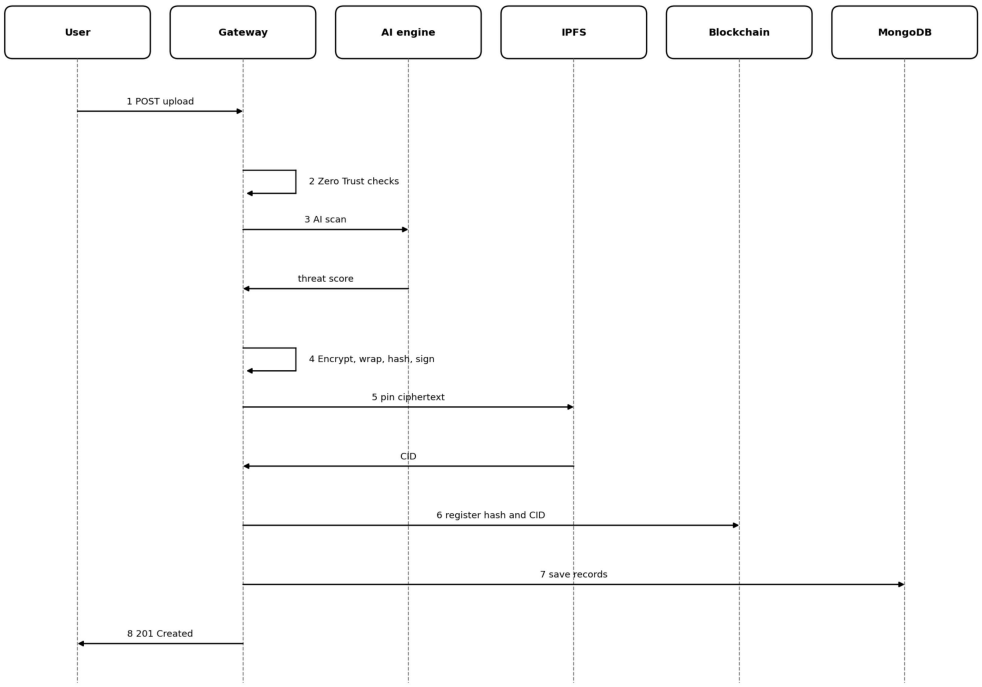


Figure 5.2: File Upload Sequence

share, and calls verifyAccess() on the blockchain registry. The ciphertext is fetched from IPFS by its CID, and the gateway checks the SHA-256 hash, unwraps the AES key and decrypts the file. For PDF files, a watermark is added. The gateway then updates download statistics and the AuditLog, and returns the file to the authorized user.

5.4 File Sharing Flow

Sharing follows two paths. In the first, the owner selects a recipient, and the gateway validates the owner's rights and the selected policy, sets the expiry and an optional download limit, adds the share entry, logs the grant, and notifies the recipient through WebSocket. In the second, the owner generates a QR link: a 32-byte random token is created and stored, and the link is displayed as a QR code. A person who opens the link can reach only that file, and the public-download endpoint checks the expiry and download limit before serving it. In both cases the policy is re-checked on every download, and grants, revocations and access attempts are logged.

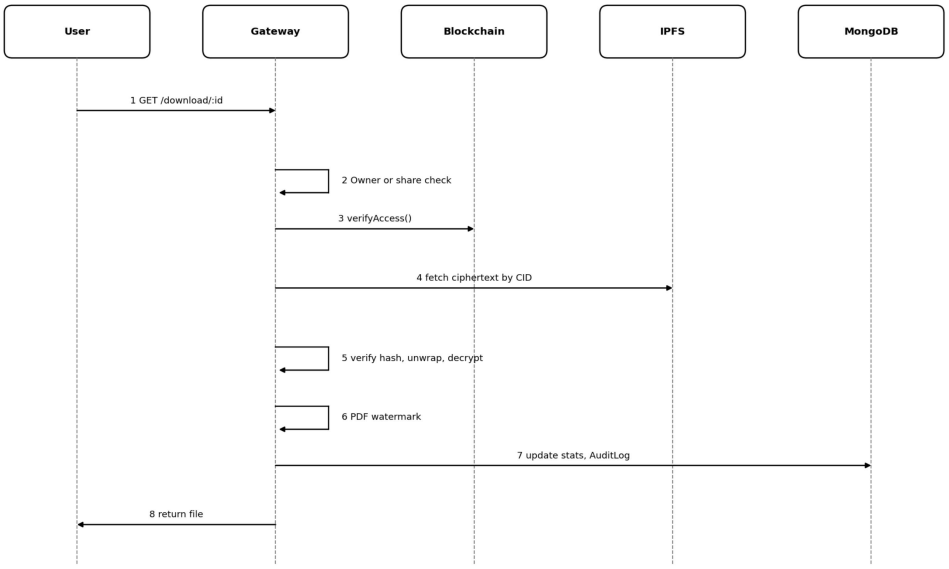


Figure 5.3: File Download Sequence

Chapter 6

WORK FLOW

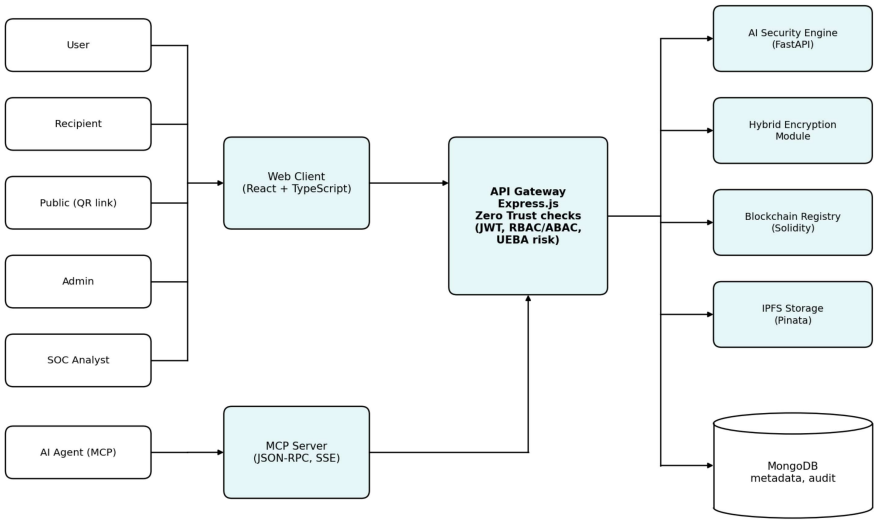


Figure 6.1: Work Flow Diagram

6.1 Description

The workflow diagram illustrates a layered file-sharing system that handles interactions from different types of actors and converts them into protected, registered and traceable files. On the left side, the diagram shows the actors: User, Recipient, Public (through a QR link), Admin, SOC Analyst and an AI agent. Each actor has a distinct responsibility. Users upload, download and share files, recipients receive shared files, a public visitor can open only a single link, admins manage users and roles, SOC analysts monitor alerts

and compliance, and AI agents analyze the system through the Model Context Protocol. None of these actors interact directly with the internal services; instead, they operate through client interfaces.

Human actors use the Web Client, which is the React and TypeScript front end. The client collects user inputs such as files, share settings and approvals and sends them as requests to the API Gateway. AI agents reach the system through the MCP server, which communicates by JSON-RPC and server-sent events and passes requests to the same gateway. The API Gateway is a single entry point into the backend. It handles request routing, authentication, validation, Zero Trust checks and risk assessment, and it ensures that requests are processed securely without exposing internal services directly to users.

Once the gateway approves a request, it forwards the request to the appropriate backend modules. The AI Security Engine inspects file content and calculates risk indicators. The Hybrid Encryption Module encrypts the file, wraps the key, computes the hash and signs it. The IPFS Storage module pins the encrypted content and returns a CID, and the Blockchain Registry records the file hash, CID and permissions so that integrity can be verified later. All operational data, including user information, file metadata, activity records and audit logs, is stored in the Database.

The overall flow is therefore clean and layered: actors interact through clients, requests are managed centrally by the gateway, processing is handled by specialized modules, and results are stored and delivered with an audit trail. This modular architecture improves maintainability, allows independent testing of each component and supports future improvements, such as running the registry on a permissioned ledger, using several pinning providers, or adding evaluated machine-learning models to the AI engine.

Chapter 7

SYSTEM REQUIREMENTS

7.1 Hardware Requirements

7.1.1 Processor

The Aegis Cyber Shield prototype runs several services at the same time, including the React development server, the Express.js gateway, the FastAPI engine, a local Hardhat blockchain node and a MongoDB instance. In addition, it performs cryptographic operations and file scanning. A multi-core processor is therefore needed. A minimum configuration of an Intel Core i5 or an equivalent AMD Ryzen 5 processor is recommended, as it provides adequate performance for development, testing and a demonstration with a small number of users.

For better performance, particularly when scanning large files, running automated tests or compiling smart contracts, a higher-end processor such as an Intel Core i7 or AMD Ryzen 7 is preferred. More cores and higher clock speeds shorten the time taken by AES and RSA operations, entropy analysis and contract tests, and multi-threading helps the system manage several requests without a noticeable loss of responsiveness.

7.1.2 Random Access Memory (RAM)

A minimum of 8 GB RAM is needed to run the full development stack. Memory is used by the Node.js and Python runtimes, the local Hardhat EVM, MongoDB, the browser-based dashboard and the temporary buffers that hold a file during scanning and encryption. If less memory is available, the services may need to be run with the mock fallbacks.

For improved performance, 16 GB RAM is recommended. The additional memory allows larger files to be scanned and encrypted in memory, reduces swapping when several services and tools run together, and gives the system headroom for the AI engine and for concurrent sessions during demonstrations.

7.1.3 Storage

A minimum of 250 GB of available storage is recommended for the source code, dependencies, container or virtual-environment files, the local blockchain data, the database, test files and logs. Solid-State Drives (SSD) are preferred over Hard Disk Drives because they provide faster read and write speeds, which speeds up file handling, builds and database access.

Encrypted files are stored on IPFS, and the local node or the mock fallback requires additional space proportional to the number and size of test files. For long-term use, a 512 GB SSD or larger is recommended, together with regular backups of the database and of the keys required for decryption. Losing the keys means losing access to the encrypted files, so key backup is a real operational need.

7.1.4 Networking Requirements

The prototype is a web-based system and needs a stable internet connection for the Pinata pinning service, for downloading dependencies and for access by remote clients. A minimum speed of 5 Mbps is required for standard use, and 10 Mbps or more is recommended when several users upload and download files at the same time, because the pinning and retrieval time depends on network speed. Clients must connect through HTTPS and WebSocket Secure so that data and alerts are protected in transit.

Latency and reliability directly affect the user experience during uploads, because several steps, such as scanning, pinning and registration, are executed in sequence. When the network is not available, the local mock fallback for IPFS allows the workflow to be demonstrated, but it does not demonstrate decentralized availability.

7.1.5 Client-Side Requirements

End users access the system through a web browser and no specialized software is required. Each client should have at least a dual-core processor and 4 GB RAM, and a modern, up-to-date browser such as Google Chrome, Mozilla Firefox or Microsoft Edge is required to support the dashboard, WebSocket connections and the Web Crypto features used by the interface. A Full HD display (1920×1080) is recommended for the SOC dashboard, charts and threat map, and a stable internet connection is needed for real-time alerts.

7.2 Software Requirements

7.2.1 Operating System

The system is designed to be cross-platform. Development and demonstration are possible on Windows 10/11 and on Linux distributions such as Ubuntu 22.04 LTS. A Linux-based server is preferred for deployment because of its stability, security and efficient handling of web server and database workloads, while Windows is convenient for development, testing and debugging.

7.2.2 Frontend Technologies

The frontend is developed with React 18 and TypeScript, which provide reusable, strongly typed components and efficient rendering. Tailwind CSS is used for styling and responsive design, and the dashboard presents charts, a threat map and live alert feeds. Communication with the gateway uses HTTPS for requests and WebSocket Secure for real-time events. Toast notifications are used for user feedback, and QR codes are generated for share links.

7.2.3 Backend Technologies

The gateway is implemented with Node.js and Express.js in TypeScript. JSON Web Tokens are used for authentication, Multer handles file uploads, and the Node.js crypto library performs AES-256-GCM, RSA-4096-OAEP, SHA-256 and ECDSA operations.

The ws library provides WebSocket support, and the Model Context Protocol SDK exposes nine tools to AI clients. A RESTful API structure keeps the frontend and backend modular and independent.

7.2.4 AI Engine

The AI security engine is written in Python 3.9 or later using FastAPI. It uses scikit-learn together with rule-based analyzers for malware heuristics, entropy analysis, personal-data detection, steganography detection and behaviour scoring. The engine is deployed as a separate service (development port 8000) so that it can be scaled and tested independently of the gateway.

7.2.5 Blockchain and Storage

Smart contracts are written in Solidity and are compiled, tested and deployed with Hardhat. The ethers.js library connects the gateway to the contracts, and a local Hardhat EVM (port 8545) is used for development, with an in-memory ledger as a fallback. File storage uses IPFS through the Pinata pinning service, with a local mock fallback. Metadata is stored in MongoDB (port 27017), with a development mock available when a database server is not installed.

7.2.6 Development and Deployment Tools

Development uses Visual Studio Code as the editor, Git and GitHub for version control, Postman for testing API endpoints, and Hardhat's test framework for smart contracts. Test scripts and logs record the results of each test case. For deployment, the services can be run on a local server for demonstration, or on cloud platforms such as AWS, Microsoft Azure or Google Cloud. Containerization with Docker, and a reverse proxy such as Nginx for TLS termination, can be used to give consistent environments. Secrets such as keys and tokens must not be hard-coded and are supplied through environment configuration.

Chapter 8

EXPECTED RESULT

8.1 Expected result

The proposed system is expected to give users a file-sharing workflow in which every file is verified, screened, encrypted, stored and registered before it is shared. An upload should produce a CID and a blockchain transaction reference, and the file hash should be available for later comparison. A download by an authorized user should succeed only after the policy check and hash verification, and a download by a non-owner, or through an expired or revoked share, should be denied and recorded. A flagged file should be handled according to the configured policy, and the scan result should be preserved with its explanation. The audit trail should show a clear upload, share and download timeline, and the SOC dashboard should present alerts and compliance scores. The results of the prototype will be reported through test evidence and are not intended to establish enterprise-scale security or compliance.

8.2 Implementation Progress

Implementation progress at the time of Phase 1 is summarized in Table 8.1. The percentages are provisional estimates for the eight modules and the overall average is about 70%. They are to be validated against the code, screenshots, logs and demonstrable tests before the review, and should not be read as measured results.

Module	Components	Progress
Zero Trust gateway	JWT, IP, session, RBAC/ABAC, UEBA	75%
Hybrid encryption	AES-256-GCM, RSA-4096, SHA-256, ECDSA	70%
AI security engine	Malware, entropy, PII, stego, UEBA	70%
Smart contracts	FileRegistry.sol, DecentralizedID.sol	65%
IPFS storage	Pinata pinning, local mock fallback	65%
SOC dashboard	Live alerts, charts, scorecards	70%
MCP tools	Nine tools over JSON-RPC and SSE	65%
Provenance	FileActivity, GDPR, ISO and NIST scoring	75%
Overall	Average of the eight modules	**about 70%**

Table 8.1: Module progress (provisional estimates)

8.3 System Implementation Results

8.3.1 Web Interface Implementation

The following screens illustrate the web interface and the main functionalities of the implemented prototype. The images shown here are placeholders and are to be replaced by the actual screenshots of the running system.

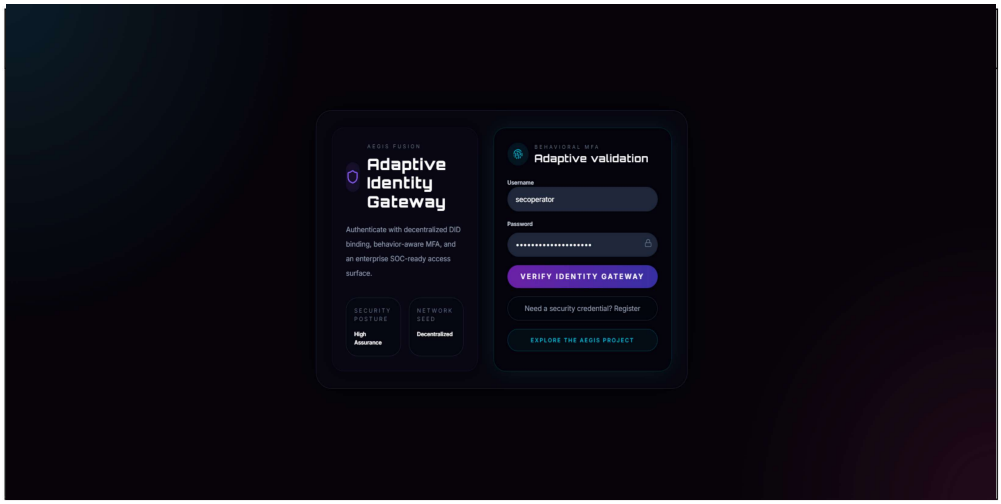


Figure 8.1: Login and Registration Page

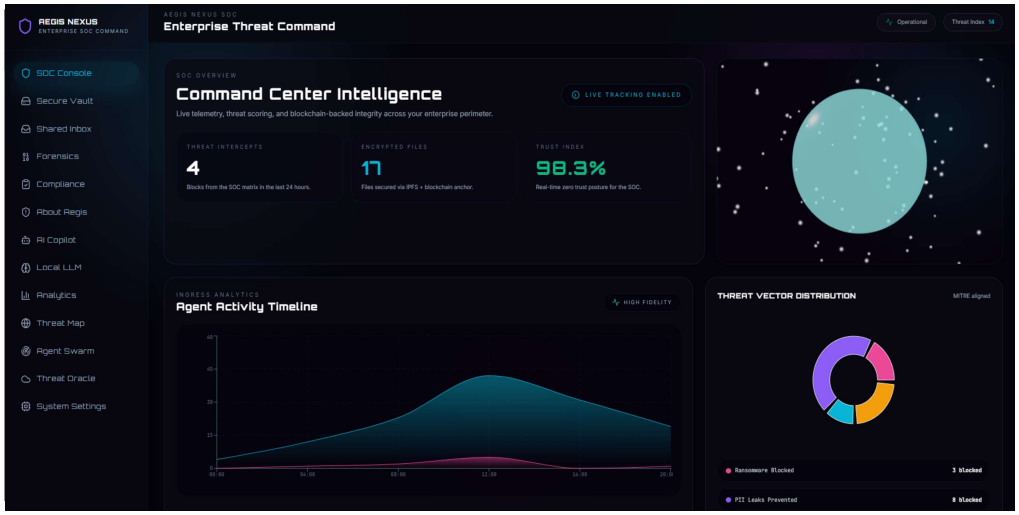


Figure 8.2: User Dashboard

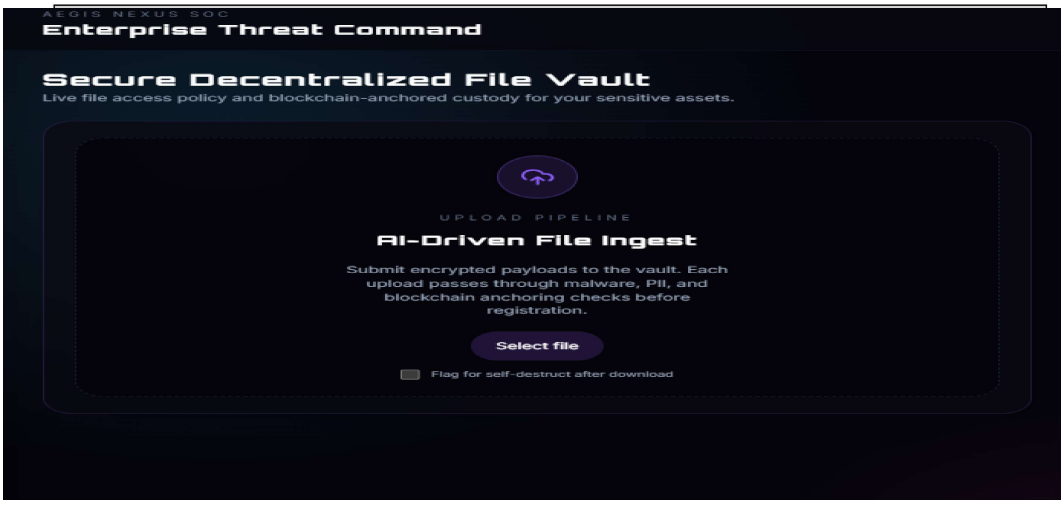


Figure 8.3: File Upload with AI Scan Result

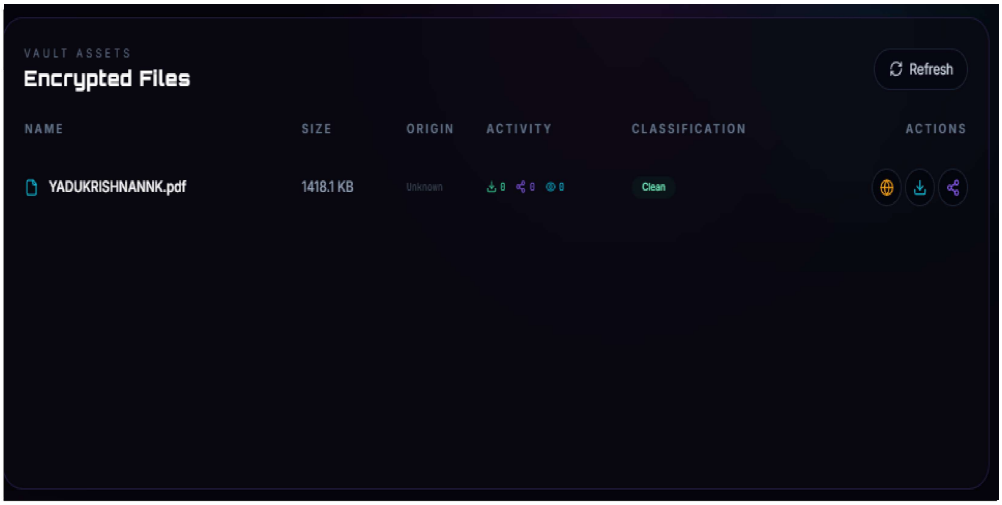


Figure 8.4: Encrypted File List with CID and Hash

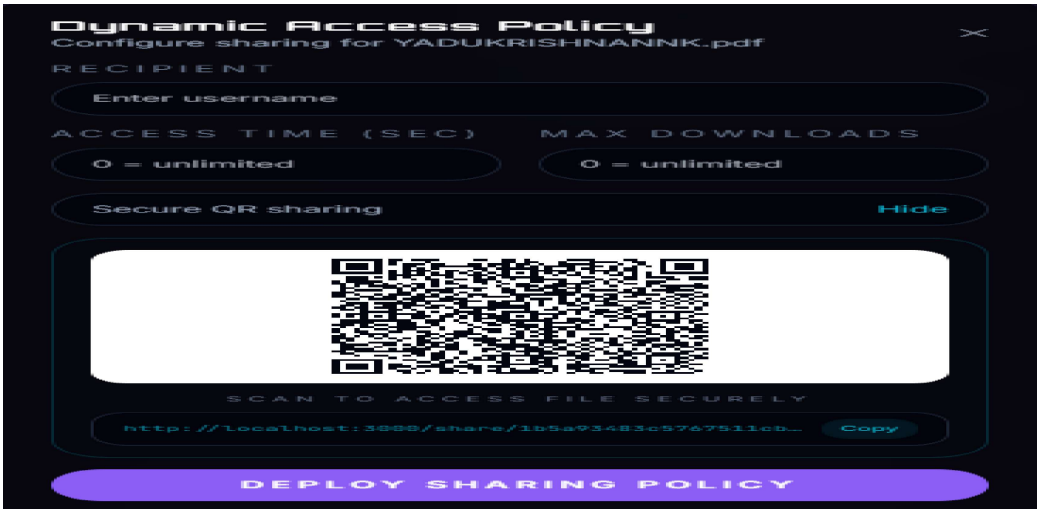


Figure 8.5: Share Dialog with Expiry and QR Link

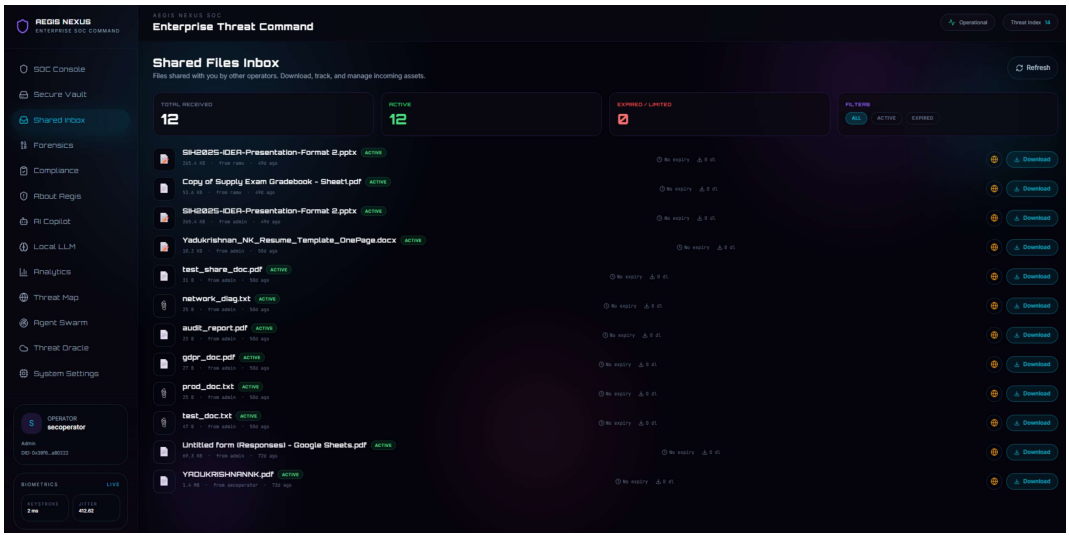


Figure 8.6: Shared Inbox

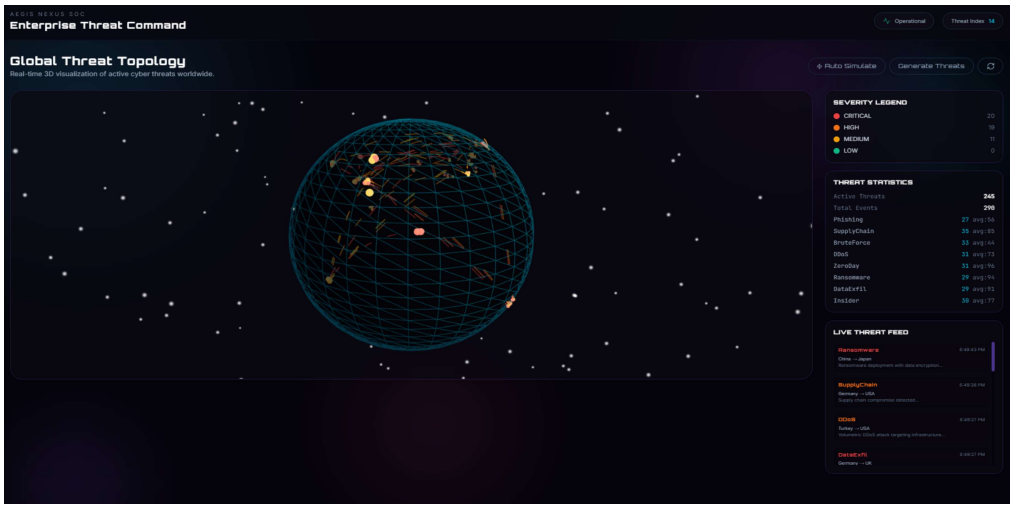


Figure 8.7: SOC Dashboard and Threat Map

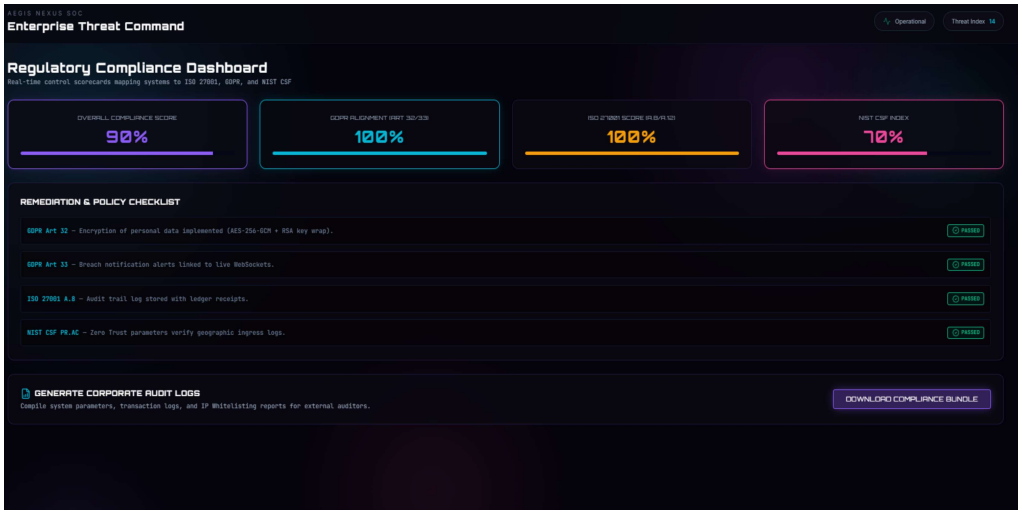


Figure 8.8: Provenance Timeline and Compliance Scorecard

8.4 Risks and Limitations

The main risks identified for the project and their mitigations are as follows.

- **IPFS availability:** content can be unavailable if no node provides it. Pinning is

used and retrieval is tested.

- **AI false positives and negatives:** transparent rule-based indicators are used, and detection limitations are reported.
- **Key loss or exposure:** access to keys is restricted, secrets are not hard-coded and the key life cycle is documented.
- **Blockchain privacy:** personal information and raw file content are kept off-chain.
- **Smart-contract defects:** access conditions are tested locally, and no production audit or certification is implied.
- **Integration complexity:** small end-to-end test cases and stable mock fallbacks are used where needed.
- **Prototype scope:** a working academic demonstration does not establish enterprise-scale security or compliance.

Chapter 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

File sharing is a high-value target, and centralized platforms depend heavily on a single server and on the practices of administrators. Aegis Cyber Shield addresses this problem by combining Zero Trust access control, AI-assisted threat screening, hybrid encryption, IPFS storage and blockchain integrity references in one file-sharing workflow. The literature survey of 30 sources supports each individual technology and, at the same time, highlights practical trade-offs in detection quality, key management, availability and auditability.

The main contribution of the project is an integrated and testable workflow, not the invention of the underlying technologies. Phase 1 has established the requirements, the five-layer architecture, the upload, download and sharing flows, the data models and the initial module implementation. The design keeps file contents encrypted and off-chain, records minimal integrity references on the blockchain, re-checks policy at every download, and produces a provenance record for each operation.

The work is not complete. The module progress is a provisional estimate of about 70%, which is to be validated by code, logs and tests. The next steps are to complete end-to-end integration, to test negative paths, to measure results, and to update the documentation from evidence. Decentralization alone does not guarantee confidentiality, availability or correct authorization, and these controls must still be implemented carefully and tested.

9.2 Future Scope of Work

The system can be further improved and expanded in the following ways:

- **Evaluation of the AI Engine on Benchmark Data**
  - Evaluate the detection modules on labelled datasets such as CICIDS2017 and on safe, labelled file samples.
  - Measure precision, recall and the false-positive rate, and tune the risk thresholds.
  - Add evaluated machine-learning models alongside the rule-based checks.
- **Stronger Key Management**
  - Move key storage to a hardware security module or a managed key service.
  - Add key rotation, recovery and revocation procedures.
  - Document the complete key life cycle in line with NIST SP 800-57.
- **Post-Quantum Readiness**
  - Replace the simulated Kyber-1024 key encapsulation with a deployed ML-KEM implementation based on NIST FIPS 203.
  - Evaluate hybrid classical and post-quantum key wrapping.
- **Production-Grade Blockchain and Storage**
  - Deploy the registry on a test network or on a permissioned ledger such as Hyperledger Fabric.
  - Use multiple pinning providers or a private IPFS cluster to improve availability.
  - Perform an independent audit of the smart contracts.
- **Scalability and Performance Optimization**
  - Support streaming encryption and chunked uploads for very large files.
  - Register hashes asynchronously and batch blockchain transactions.

  - Carry out load testing with many concurrent users.

- **Usability and Platform Extension**

  - Develop mobile applications for sharing and for receiving alerts.
  - Improve the explanation of scan outcomes and sharing status for non-technical users.
  - Integrate with existing identity providers and organizational directories.

Bibliography

[1] S. Nakamoto (2008). Bitcoin: A peer-to-peer electronic cash system. White paper. https://bitcoin.org/bitcoin.pdf

[2] G. Wood (2014). Ethereum: A secure decentralised generalised transaction ledger. Yellow Paper. https://ethereum.github.io/yellowpaper/paper.pdf

[3] J. Benet (2014). IPFS: Content addressed, versioned, P2P file system. arXiv:1407.3561. https://arxiv.org/abs/1407.3561

[4] S. Haber and W. S. Stornetta (1991). How to time-stamp a digital document. Journal of Cryptology, 3(2). https://doi.org/10.1007/BF00196791

[5] E. Androulaki et al. (2018). Hyperledger Fabric: A distributed operating system for permissioned blockchains. EuroSys '18. https://doi.org/10.1145/3190508.3190538

[6] M. S. Ali et al. (2019). Applications of blockchains in the Internet of Things: A comprehensive survey. IEEE Communications Surveys & Tutorials.

[7] C. Alevizos et al. (2022). Blockchain-enabled intrusion detection and prevention system of APTs within zero trust architecture. IEEE Access, 10, 89270–89288. https://doi.org/10.1109/ACCESS.2022.3200165

[8] W. Diffie and M. E. Hellman (1976). New directions in cryptography. IEEE Transactions on Information Theory, 22(6). https://doi.org/10.1109/TIT.1976.1055638

[9] R. L. Rivest, A. Shamir and L. Adleman (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2). https://doi.org/10.1145/359340.359342

[10] C. E. Shannon (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3). https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

[11] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP. https://doi.org/10.5220/0006639801080116

[12] G. Apruzzese et al. (2023). The role of machine learning in cybersecurity. ACM Digital Threats: Research and Practice, 4(1). https://doi.org/10.1145/3545574

[13] T. Bass (2000). Intrusion detection systems and multisensor data fusion. Communications of the ACM, 43(4).

[14] F. Monrose and A. D. Rubin (2000). Keystroke dynamics as a biometric for authentication. Future Generation Computer Systems, Security on the Web special issue.

[15] M. Vielberth et al. (2020). Security operations center: A systematic study and open challenges. IEEE Access, 8, 227756–227779. https://doi.org/10.1109/ACCESS.2020.3045514

[16] B. Carrier and E. H. Spafford (2003). Getting physical with the digital investigation process. CERIAS Technical Report 2003-29.

[17] S. Rose, O. Borchert, S. Mitchell and S. Connelly (2020). Zero Trust Architecture. NIST SP 800-207. https://doi.org/10.6028/NIST.SP.800-207

[18] NIST (2024). The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29. https://doi.org/10.6028/NIST.CSWP.29

[19] ISO/IEC (2022). ISO/IEC 27001:2022, Information security management systems: Requirements. https://www.iso.org/standard/27001

[20] European Union (2016). Regulation (EU) 2016/679 (GDPR), Article 32. https://eur-lex.europa.eu/eli/reg/2016/679/oj

[21] NIST (2001). FIPS 197: Advanced Encryption Standard (AES). https://doi.org/10.6028/NIST.FIPS.197

[22] M. Dworkin (2007). NIST SP 800-38D: Recommendation for block cipher modes of operation: GCM and GMAC. https://doi.org/10.6028/NIST.SP.800-38D

[23] K. Moriarty, B. Kaliski, J. Jonsson and A. Rusch (2016). RFC 8017: PKCS #1 v2.2, RSA cryptography specifications. https://doi.org/10.17487/RFC8017

[24] NIST (2015). FIPS 180-4: Secure Hash Standard (SHS). https://doi.org/10.6028/NIST.FIPS.180-4

[25] NIST (2023). FIPS 186-5: Digital Signature Standard (DSS). https://doi.org/10.6028/NIST.FIPS.186-5

[26] NIST (2024). FIPS 203: Module-lattice-based key-encapsulation mechanism standard. https://doi.org/10.6028/NIST.FIPS.203

[27] IBM Security (2023). Cost of a Data Breach Report 2023. https://www.ibm.com/reports/data-breach

[28] MITRE. MITRE ATT&CK knowledge base. https://attack.mitre.org

[29] Model Context Protocol. Specification. https://modelcontextprotocol.io/specification

[30] OWASP Foundation (2021). OWASP Top 10:2021. https://owasp.org/Top10/



Department of Computer Science and Engineering
Vidya Academy of Science & Technology
Thalakkottukara, Thrissur - 680 501
(http://www.vidyaacademy.ac.in)